KONGU ENGINEERING COLLEGE

DEPARTMENT OF INFORMATION TECHNOLOGY

SALESFORCE DEVELOPER PROJECT DOCUMENTATION

Revolutionizing Agriculture with AgriEdge Or-Mange Ltd: A Salesforce-Driven Order Management Solution

PROJECT DOCUMENTATION

Prepared by

VIDHYA A

24ITR165

Table of Contents

• Project Overview

• Objectives

• Phase 1 – Requirement Analysis & Planning

•Phase 2 – Salesforce Development: Backend & Configurations

• Phase 3 – UI/UX Development & Customization

•Phase 4 – Data Migration, Testing & Security

•Phase 5 – Deployment, Documentation & Maintenance

•Screenshots

•Features Not Implemented in the Current Scope

•Future Enhancements

•Conclusion

1. Project Overview

AgriEdge Or-Mange Ltd is implemented as a Salesforce-driven order management solution designed to organize customer orders, order items, inventory information, and shipment tracking in a single CRM environment. The solution uses custom Salesforce objects, relationships, formulas, profiles, roles, field-level security, Process Builder automation, Apex classes and triggers, and Apex test classes. The implementation is intended to improve order visibility, automate important order-processing activities, and provide a controlled environment for users with different responsibilities.

The documented implementation reflects the components actually developed in the project. Features that were not implemented are explicitly marked as not implemented rather than being presented as completed.

2. Objectives

- Create a centralized Salesforce solution for managing customer orders and related order items.
- Maintain inventory records with automatic stock-status calculation based on stock quantity and reorder level.
- Track shipment information and shipment status against customer orders.
- Use role hierarchy, profiles, and field-level security to control access to business data.
- Automate order-related activities using Process Builder, Apex classes, and triggers.
- Calculate order-item totals and discounted order totals using Salesforce formulas.
- Develop and execute Apex test classes and achieve the required code-coverage target for the implemented Apex components.
- Provide a maintainable project structure that can be demonstrated and shared through GitHub.

3. Phase 1 – Requirement Analysis & Planning

3.1 Understanding Business Requirements

The business requirement is to manage the order lifecycle in Salesforce while keeping customer, order, order-item, inventory, and shipment information connected. Different users require different levels of access, and repetitive activities such as task creation and order-status handling should be automated.

3.2 Project Scope

Area	Scope
Customer management	Account used as the customer reference for AgriEdge Orders.
Order management	Order status, date, amount, payment status, shipping address and discounted total.
Order items	Product, quantity, unit price and calculated total price.
Inventory	Product, stock quantity, reorder level, warehouse location and calculated stock status.
Shipment	Order, tracking number, carrier and shipment status.
Security	Roles, profiles and field-level security.
Automation	Process Builder, Apex classes and Apex triggers.

Testing	Apex test class covering implemented Apex functionality.

3.3 Data Model

The data model connects AgriEdge Order with Account through Customer__c. AgriEdge Order Item uses Order__c to reference AgriEdge Order and Product__c to reference Product. AgriEdge Inventory references Product through Product__c. AgriEdge Shipment references AgriEdge Order through Order__c.

Object	Important Relationships / Purpose
AgriEdge Order	Customer__c → Account; parent record for order processing.
AgriEdge Order Item	Order__c → AgriEdge Order; Product__c → Product.
AgriEdge Inventory	Product__c → Product; stores stock and reorder information.
AgriEdge Shipment	Order__c → AgriEdge Order; stores carrier and shipment status.

3.4 Security Model

The project uses a role hierarchy and custom profiles. The planned hierarchy is Kongu Engineering college → CEO, with Sales Representative and Warehouse Manager under CEO, and Finance Team under Sales Representative. Profiles Platform 2 and Platform 3 were configured with object permissions according to the project requirements. Field-level security was also configured for selected AgriEdge Order fields.

4. Phase 2 – Salesforce Development: Backend & Configurations

4.1 Salesforce Environment & Development Workflow

The solution was configured in a Salesforce Developer Edition environment using Lightning Experience, Setup, Developer Console, custom objects, fields, profiles, roles, Process Builder, Apex classes, triggers, and Apex tests.

4.2 Custom Objects and Fields

Object	Key Fields / Configuration
AgriEdge Order	Order Number (Auto Number); Customer__c; Order_Status__c; Order_Date__c; Total_Amount__c; Payment_Status__c; Shipping_Address__c; Discounted_Total__c.
AgriEdge Order Item	Order Item Number (Auto Number); Order__c; Product__c; Quantity__c; Unit_Price__c; Total_Price__c.
AgriEdge Inventory	Inventory Number (Auto Number); Product__c; Stock_Quantity__c; Reorder_Level__c; Warehouse_Location__c; Stock_Status__c.
AgriEdge Shipment	Shipment Number (Auto Number); Order__c; Tracking_Number__c; Carrier__c; Status__c.

4.3 Formula Fields

Field	Formula	Purpose
Discounted_Total__c	Total_Amount__c - (Total_Amount__c * 0.1)	Calculates a 10% discounted order total.
Total_Price__c	Quantity__c * Unit_Price__c	Calculates the order-item total.
Stock_Status__c	IF(Stock_Quantity__c <= Reorder_Level__c, "Low", "Sufficient")	Shows whether stock is low or sufficient.

4.4 Process Builder Automation

A Process Builder named 'Order Task Creator' was configured for AgriEdge Order records. The process starts when an AgriEdge Order record is created, checks the Order Status condition for 'New', and invokes the Apex action 'Create Task for New Order' using the OrderTaskCreator Apex class.

4.5 Apex Classes and Triggers

Component	Purpose
OrderTaskCreator	Invocable Apex action that creates a high-priority task for a new order and assigns it to users with the Platform 1 profile.
OrderStatusUpdater	Changes eligible New orders to Processing.
OrderTotalUpdater	Aggregates AgriEdge Order Item totals and updates the parent order amount and payment status.
OrderItemTrigger	Recalculates the related order total after order-item insert, update, delete or undelete.
OrderEmailSender	Loads order information for email-related processing; current implementation contains the order query and placeholder email logic.
OrderPaymentStatusTriggers	Updates payment status based on whether the order total is greater than zero.
AgriEdgeOrderShipmentHelper	Handles order-status/payment-status-related shipment and order-item processing.
AgriEdgeOrderTriggerHelper	Invokes shipment processing after order update and contains a recursion guard.
AgriEdgeOrderTrigger	After-update trigger on AgriEdge Order.

No asynchronous Apex class is included in the implemented scope documented here.

5. Phase 3 – UI/UX Development & Customization

5.1 Lightning App

The AgriEdge Lightning app was created to provide navigation to the main business areas, including AgriEdge Orders, AgriEdge Order Items, AgriEdge Inventories, AgriEdge Shipments, Reports and Dashboards.

5.2 Page Layouts and Lightning Pages

Document the page layouts and Lightning Pages that are actually configured in the org. Dynamic Forms and LWC should only be listed as implemented if they were created.

Current implementation record: No LWC was developed for the documented scope.

5.3 User Management

User / Role	Profile	Purpose
John Production Engineer Sandbox 1	Platform 1	Sales Representative role / primary business user.
Quality Inspector Mike	Platform 2	Warehouse Manager role.
Plant Manager Albert	Platform 3	Finance Team role.

6. Phase 4 – Data Migration, Testing & Security

6.1 Data Loading

The project demonstration uses Salesforce records created during configuration and testing. A Data Import Wizard or Data Loader migration process was not documented as part of the implemented scope.

6.2 Field History, Duplicate and Matching Rules

These features should be documented only if configured in the org. They are not included as completed components in this project documentation.

6.3 Profiles and Field-Level Security

Platform 2 was configured with Read/Create/Edit access for AgriEdge Inventory, Account and AgriEdge Shipment, and read-only access for AgriEdge Order and AgriEdge Order Items. Platform 3 was configured with Read/Create/Edit access for AgriEdge Order and AgriEdge Order Items.

Selected AgriEdge Order fields were restricted using field-level security according to the project requirements, including Payment Status, Discounted Total and Total Amount for the specified profiles.

6.4 Apex Testing

The AgriEdgeOrderTests test class was executed after resolving trigger-recursion issues. The final documented test run reported 7 tests with 0 failures and overall code coverage of 76%.

Test Item	Expected / Observed Result
Apex test class	AgriEdgeOrderTests
Tests executed	7
Failures	0
Overall code coverage	76%
Status	Passed

6.5 Test Cases and Screenshot Evidence

The following evidence should be inserted into the final document. Each screenshot should show the input/action and the resulting output where applicable, as required by the project guidelines.

Test Case	Input / Action	Expected Output
Order creation	Create an AgriEdge Order with required fields.	Order record created; formula fields display correctly.
Discounted total	Enter Total Amount = 7000.	Discounted Total = 6300.
Order item total	Enter Quantity = 2 and Unit Price = 25.	Total Price = 50.
Inventory status	Set Stock Quantity at or below Reorder Level.	Stock Status = Low.

Process Builder task	Create an AgriEdge Order with Order Status = New.	Process invokes OrderTaskCreator and creates a task.
Order-item total update	Insert/update/delete order items.	Parent order total is recalculated.
Order trigger	Update Payment Status to Paid.	Order-status processing occurs according to the implemented trigger/helper logic.
Apex test execution	Run AgriEdgeOrderTests.	7 tests pass; 0 failures; 76% overall coverage.

7. Phase 5 – Deployment, Documentation & Maintenance

7.1 Deployment Strategy

For the current project, the configured Salesforce Developer Edition org is the development and demonstration environment. If the solution is moved to another Salesforce org, metadata deployment can be performed using an appropriate Salesforce deployment mechanism such as change sets or another approved metadata deployment method.

7.2 Maintenance and Monitoring

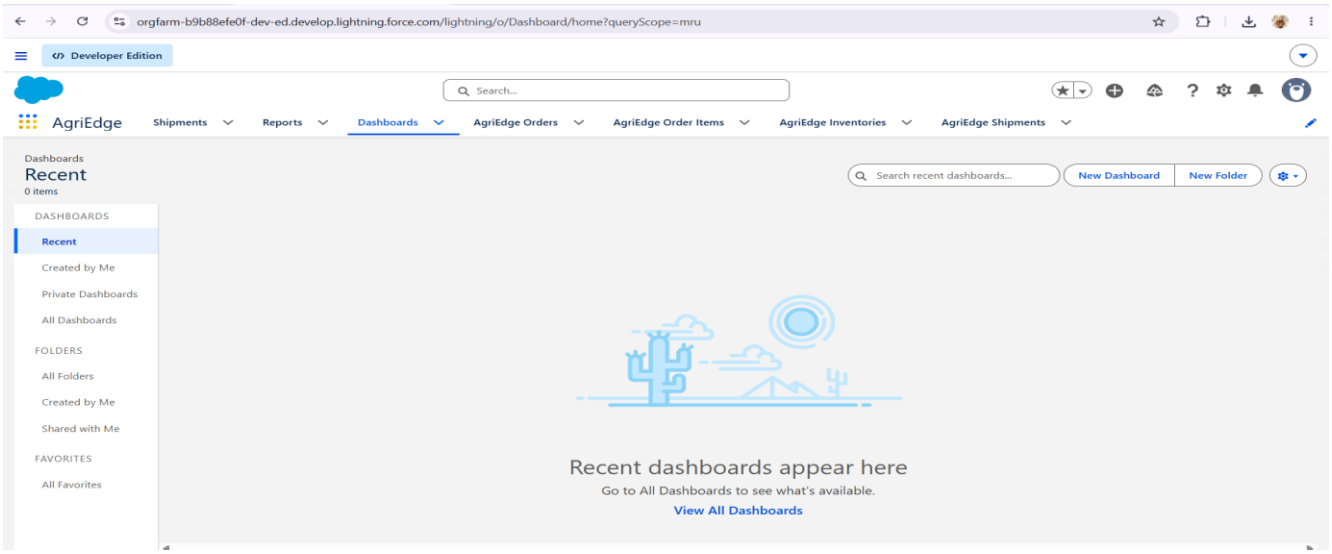
- Review Process Builder/Apex behavior after changes to order fields or status values.
- Monitor Apex test results and maintain the required code coverage.
- Review user profiles and field-level security when responsibilities change.
- Validate formula fields whenever the underlying field names or business rules change.
- Use debug logs and test data to investigate automation failures.

7.3 Troubleshooting Approach

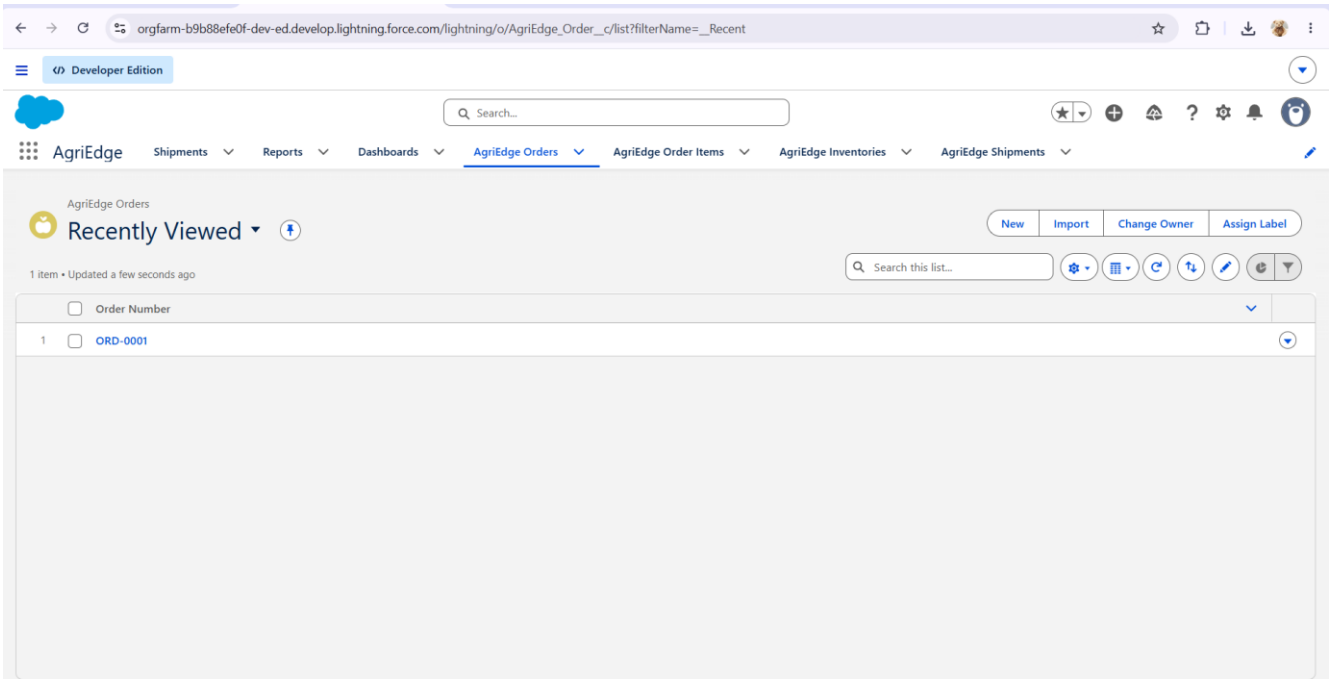
During development, trigger recursion was identified during Apex testing and resolved by adding a recursion guard in AgriEdgeOrderTriggerHelper. Apex test execution was repeated after the correction, resulting in zero failures.

8. Screenshots

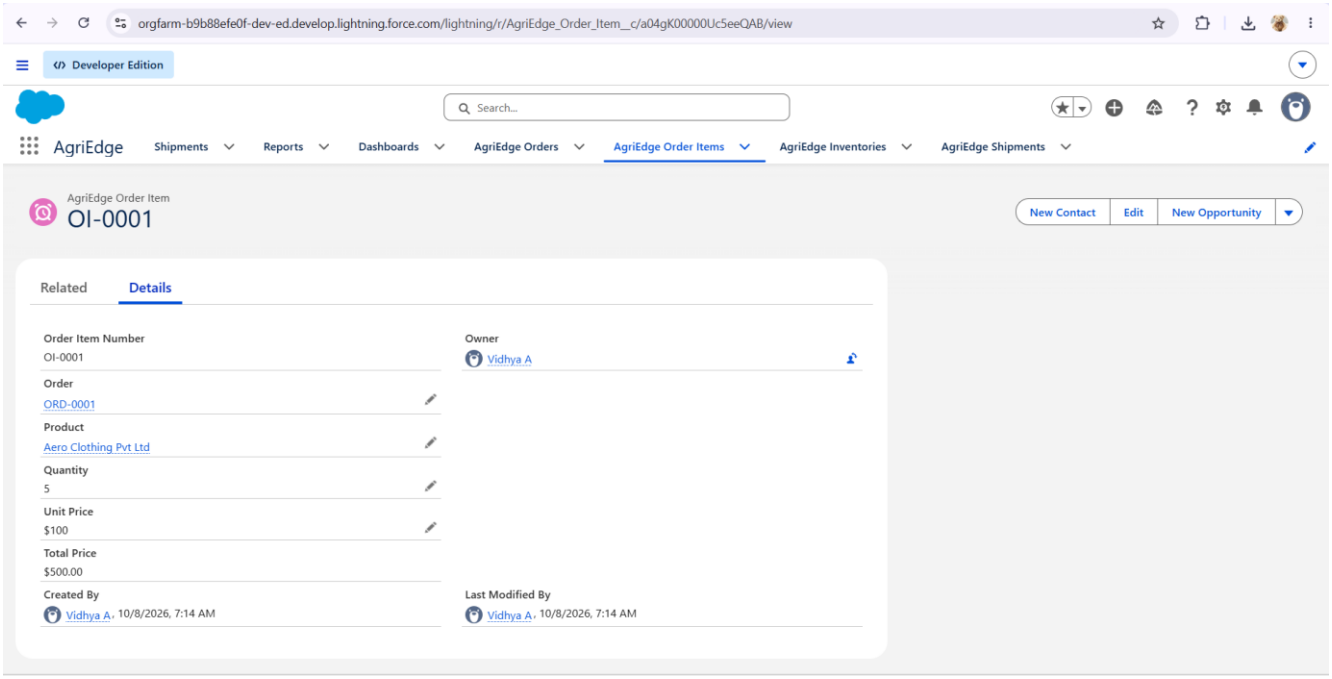
[SCREENSHOT 1 – AgriEdge App Launcher / Navigation]



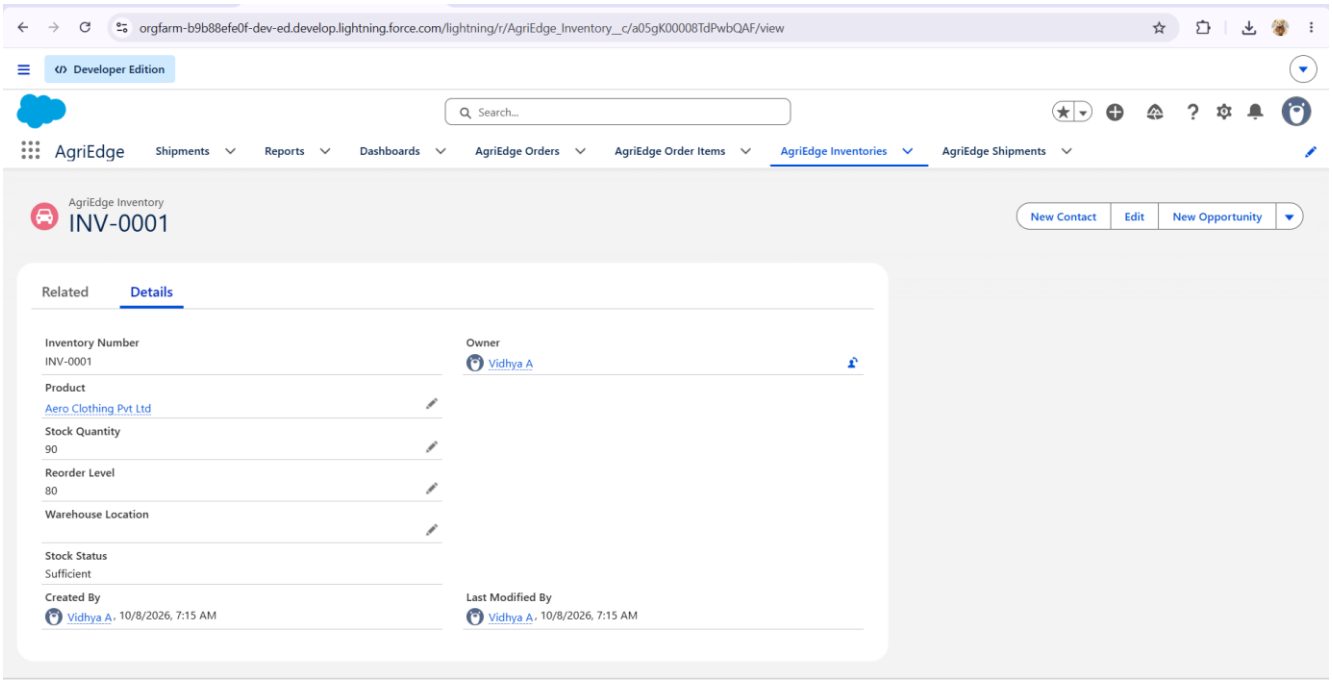
[SCREENSHOT 2 –AgriEdge Order record]



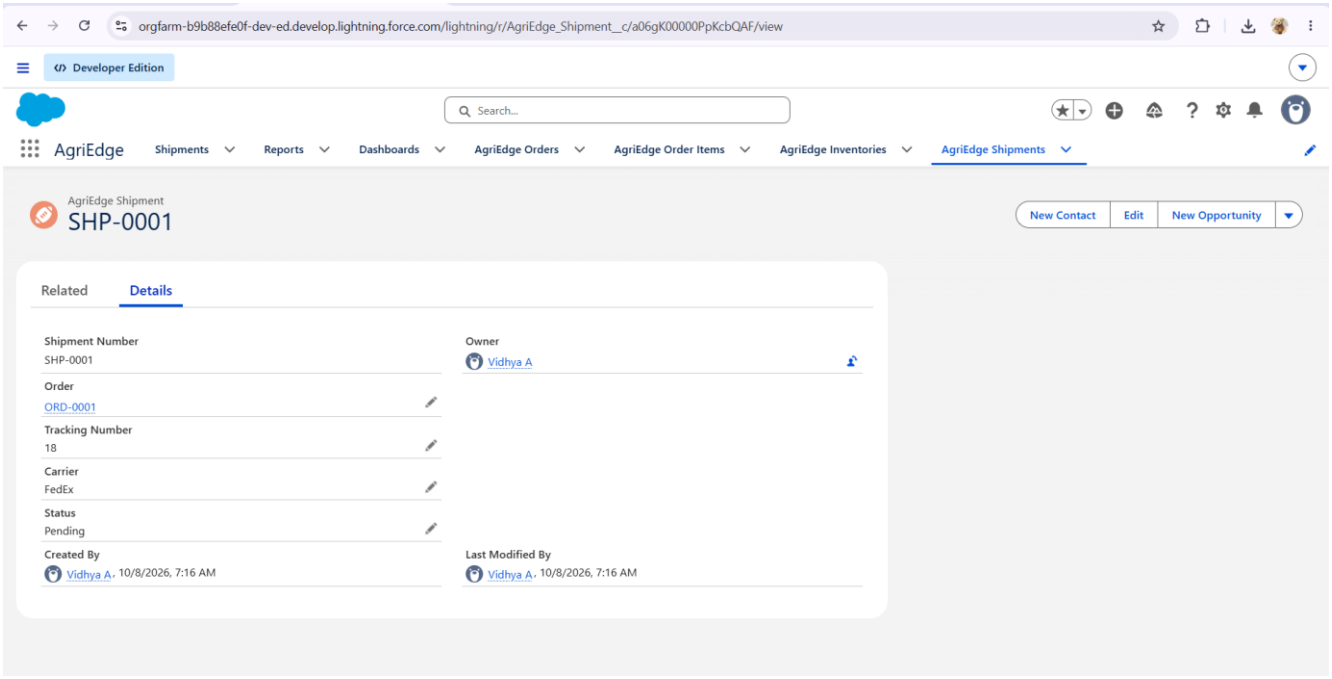
[SCREENSHOT 3 – AgriEdge Order Item Record]



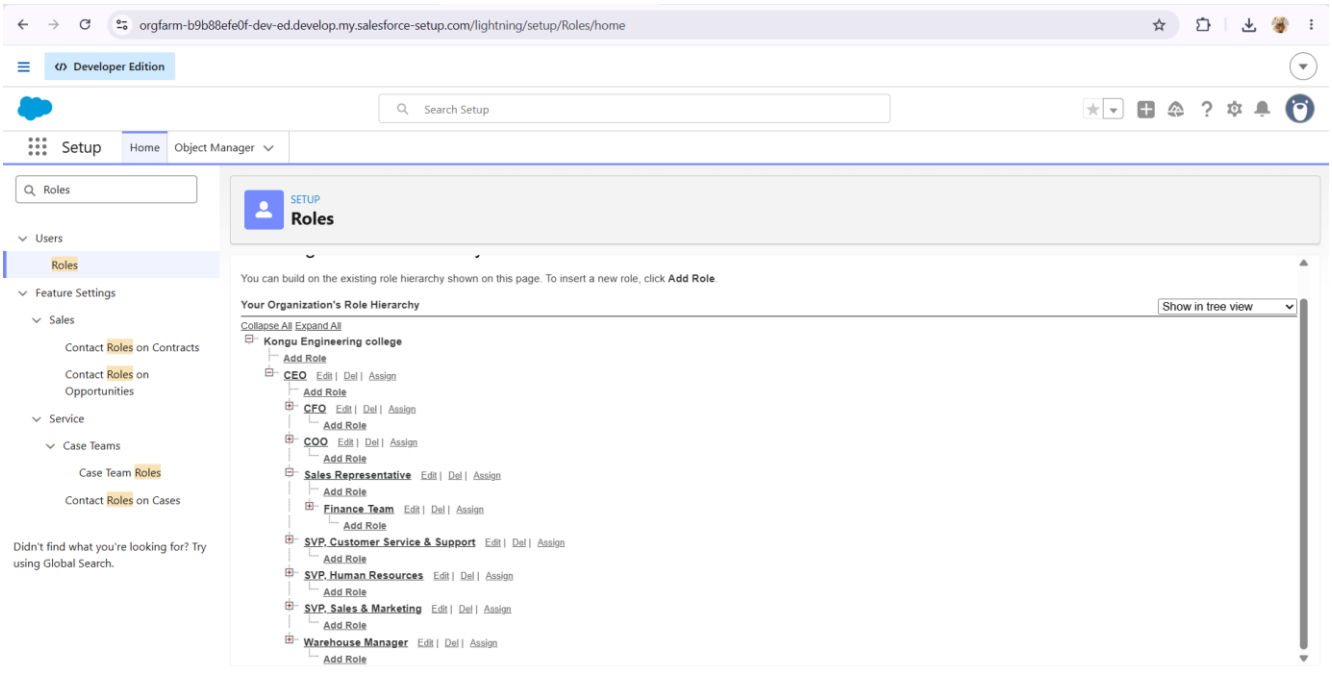
[SCREENSHOT 4 – AgriEdge Inventory Record]



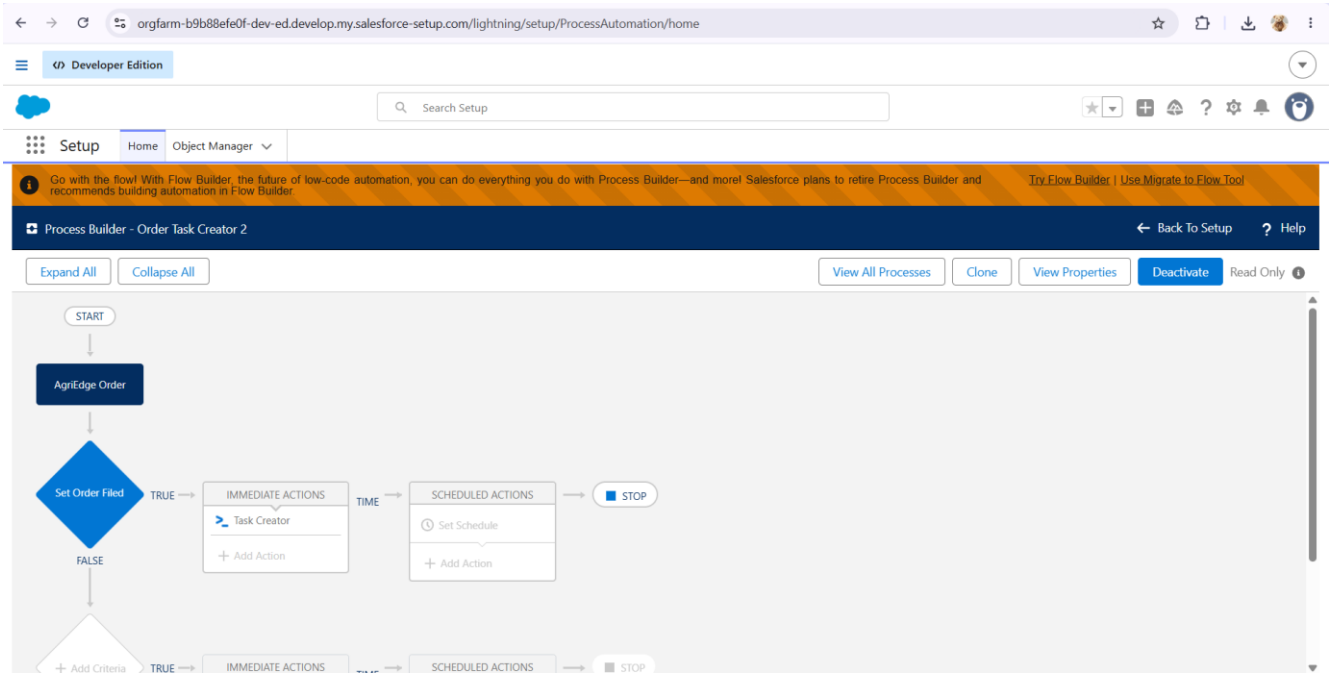
[SCREENSHOT 5 – AgriEdge Shipment Record]



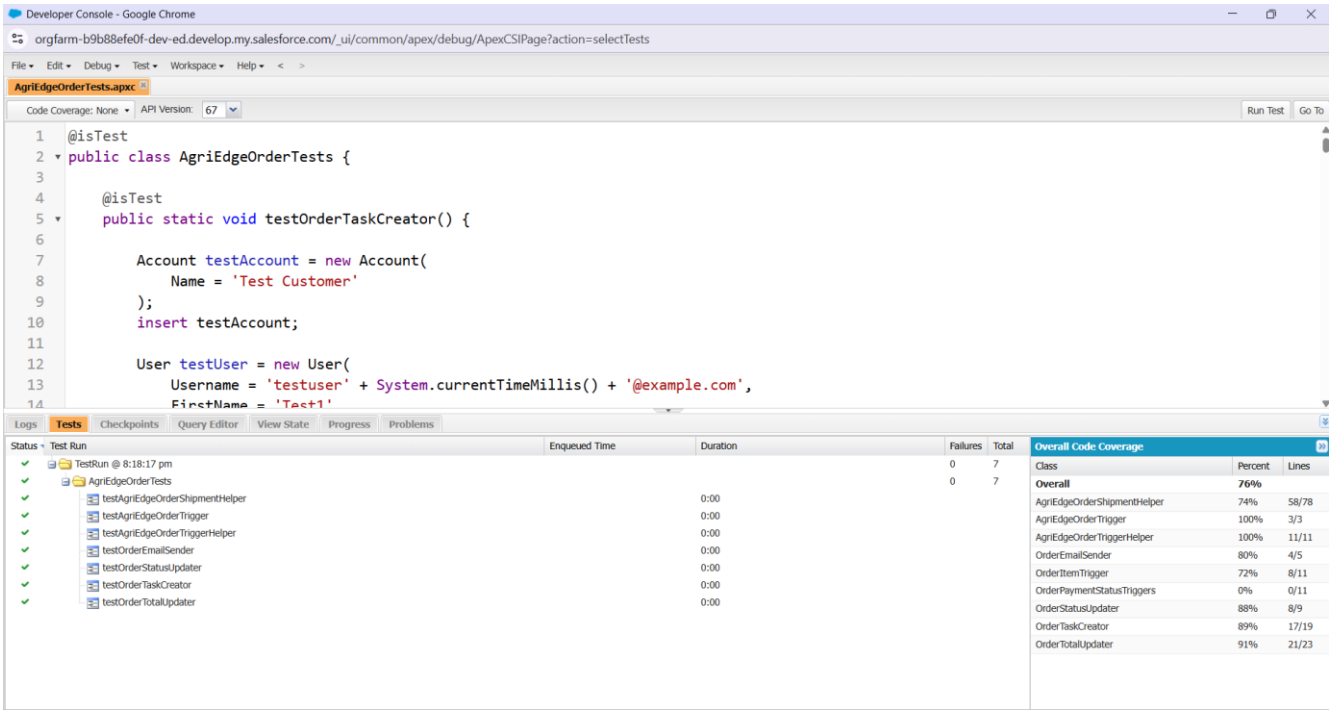
[SCREENSHOT 6 – AgriEdge Role Hierarchy]



[SCREENSHOT 7 –Order Task Creator Process Builder]



[SCREENSHOT 8 – AgriEdge Apex Test Results and Code Coverage]



.

9. Features Not Implemented in the Current Scope

To avoid inaccurate claims, the following items should not be described as completed unless they are added and tested before submission:

- Lightning Web Components (LWC).
- Asynchronous Apex.
- Approval Process.
- Workflow Rules or additional Flows beyond the documented Process Builder.
- Data Import Wizard/Data Loader migration.
- Field History Tracking, Duplicate Rules and Matching Rules.
- Permission Sets and Sharing Rules, unless separately configured.
- Reports and Dashboards, unless created and tested.

10. Future Enhancements

- Add an LWC dashboard for real-time order, inventory and shipment monitoring.
- Add chatbot integration for order-status and customer-support queries.
- Add AI-based suggestions for inventory replenishment and order prioritization.
- Introduce approval automation for exceptional or high-value orders.
- Add stronger data-quality controls using duplicate and matching rules.
- Add formal data migration and deployment pipelines for production environments.

11. Conclusion

The AgriEdge Salesforce solution provides a structured CRM-based approach to order management by connecting customers, orders, order items, inventory and shipment information. Formula fields simplify business calculations, security configuration controls access, and Process Builder plus Apex automation reduces repetitive order-processing work. The implemented Apex test class was executed successfully with seven tests, zero failures and 76% overall code coverage. The solution establishes a foundation that can be extended with analytics, LWC interfaces, AI capabilities, stronger data-quality controls and production deployment practices.